

Reverse Mapping (rmap) in Linux Kernel

Adrian Huang | May, 2022

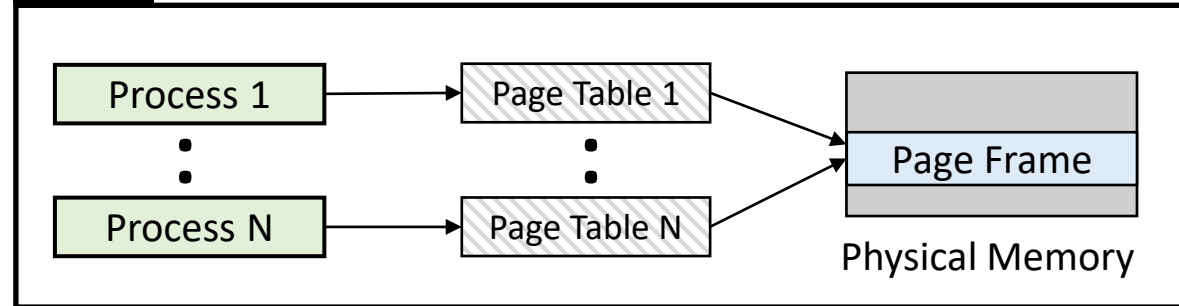
- * Based on kernel 5.11 (x86_64) – QEMU
- * SMP (4 CPUs) and 8GB memory
- * Kernel parameter: nokaslr norandmaps
- * Userspace: ASLR is disabled
- * Legacy BIOS

Agenda

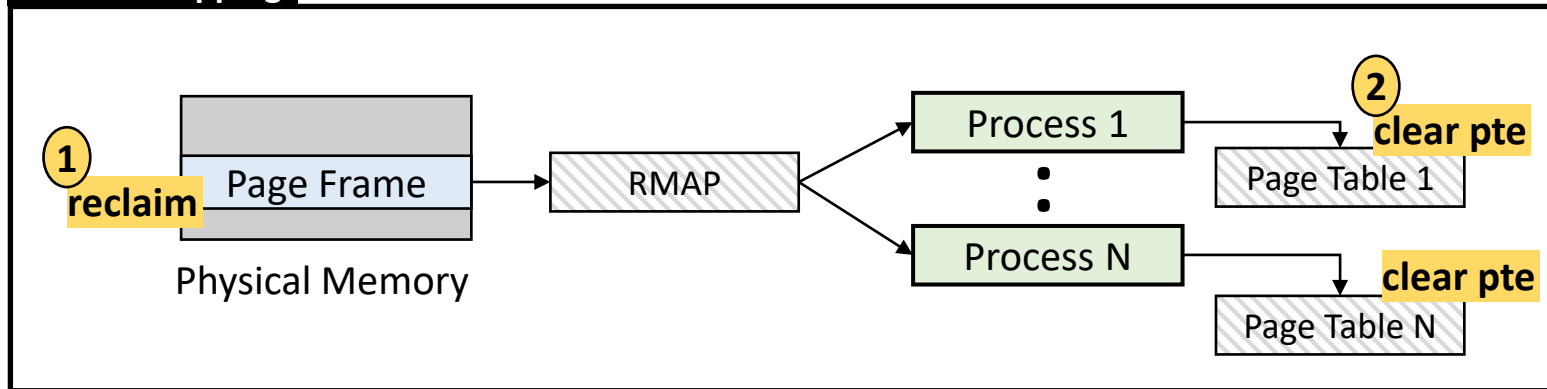
- Mapping & reverse mapping
- rmap: legacy approach vs new approach (performance improvement)
- Implementation Detail

Mapping & Reverse Mapping

Mapping



Reverse mapping



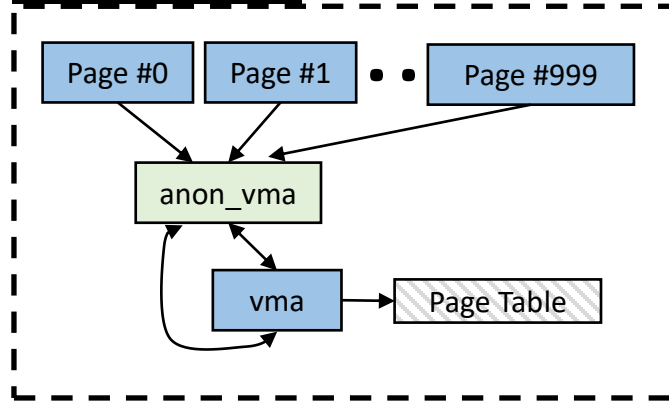
rmap – “clear pte”: check ptep_get_and_clear()

rmap: legacy approach vs new approach (performance improvement)

1. Legacy approach: 2.6.33 or earlier kernel
2. New approach: 2.6.34 or later kernel – High-level overview

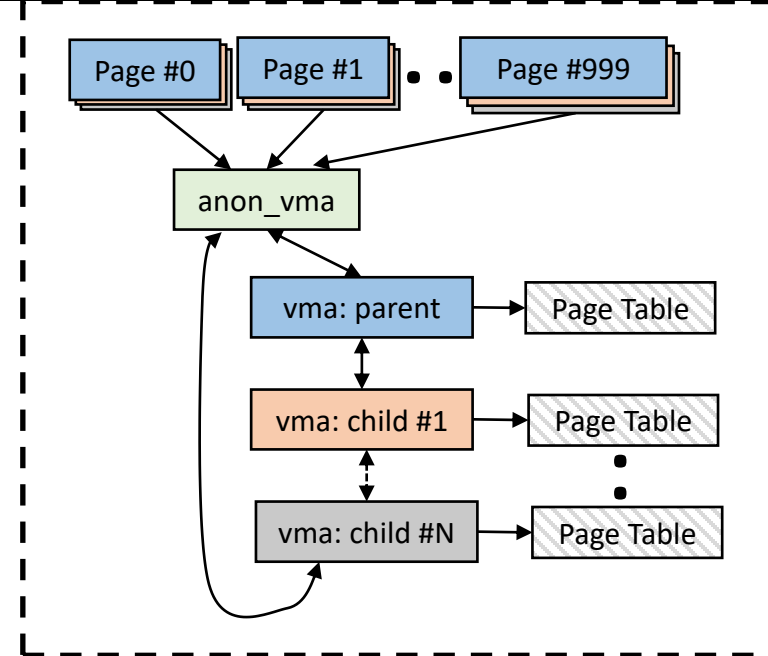
rmap: 2.6.33 or earlier kernel

Parent process



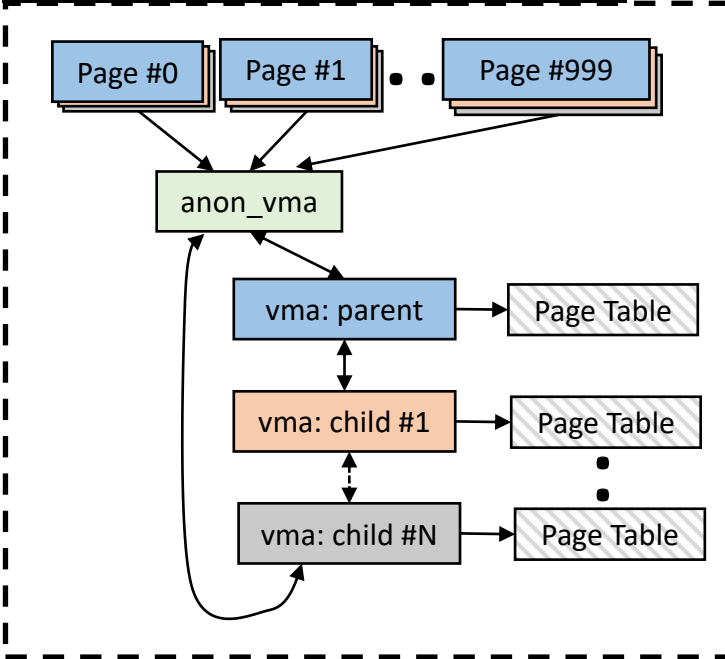
Fork #N children

Parent process & child processes: Some pages may be COWed



rmap: 2.6.33 or earlier kernel - Limitation

Parent process & child processes



Issue statement

commit 5beb49305251e5669852ed541e8e2f2f7696c53e

Author: Rik van Riel <riel@redhat.com>

Date: Fri Mar 5 13:42:07 2010 -0800

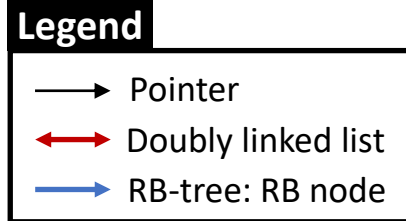
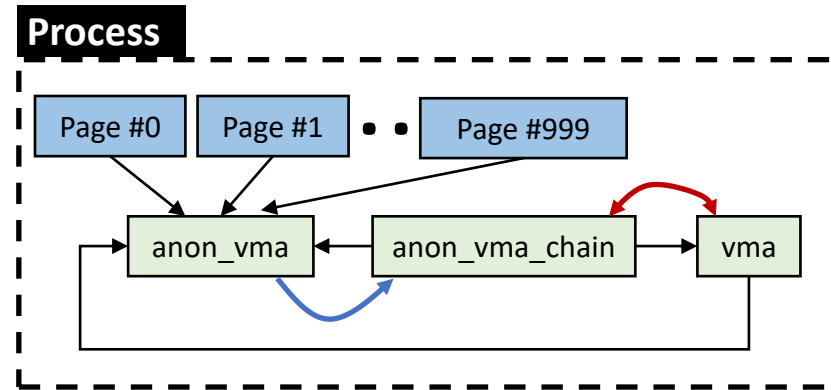
mm: change anon_vma linking to fix multi-process server scalability issue

The old anon_vma code can lead to scalability issues with heavily forking workloads. Specifically, each anon_vma will be shared between the parent process and all its child processes.

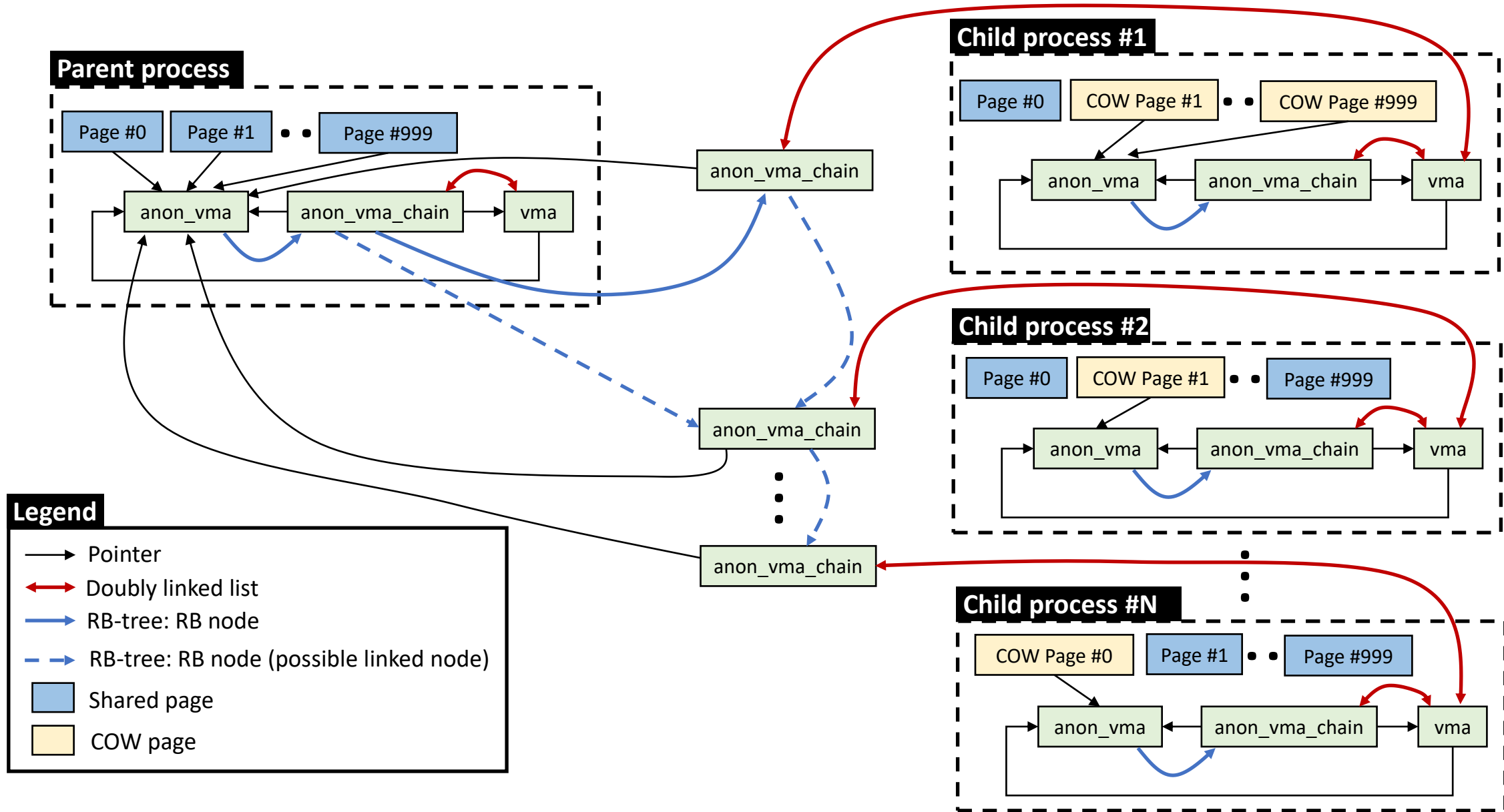
In a workload with 1000 child processes and a VMA with 1000 anonymous pages per process that get COWed, this leads to a system with a million anonymous pages in the same anon_vma, each of which is mapped in just one of the 1000 processes. However, the current rmap code needs to walk them all, leading to $O(N)$ scanning complexity for each page.

This can result in systems where one CPU is walking the page tables of 1000 processes in `page_referenced_one`, while all other CPUs are stuck on the anon_vma lock. This leads to catastrophic failure for a benchmark like AIM7, where the total number of processes can reach in the tens of thousands. Real workloads are still a factor 10 less process intensive than AIM7, but they are catching up.

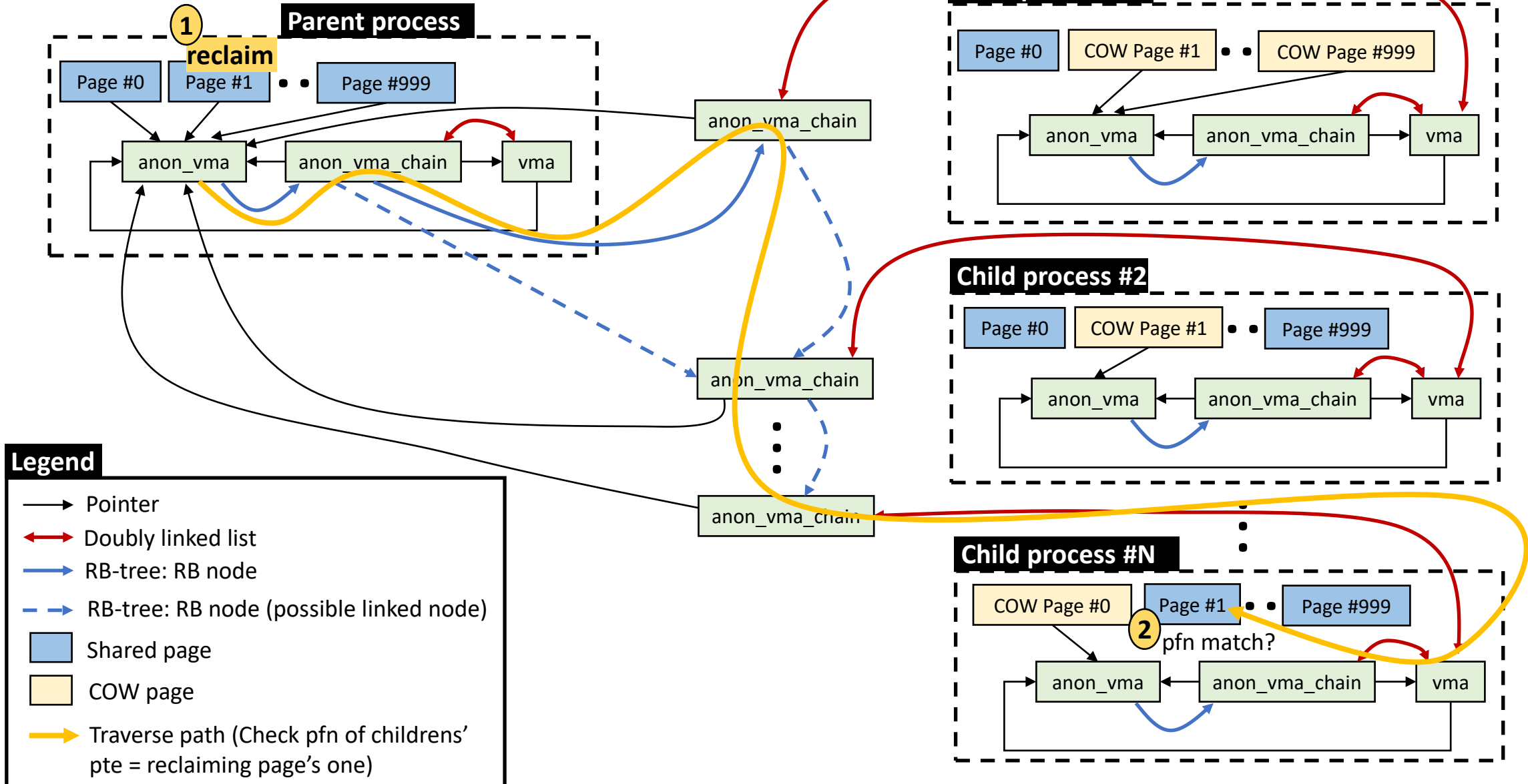
2.6.34 or later kernel – High-level overview



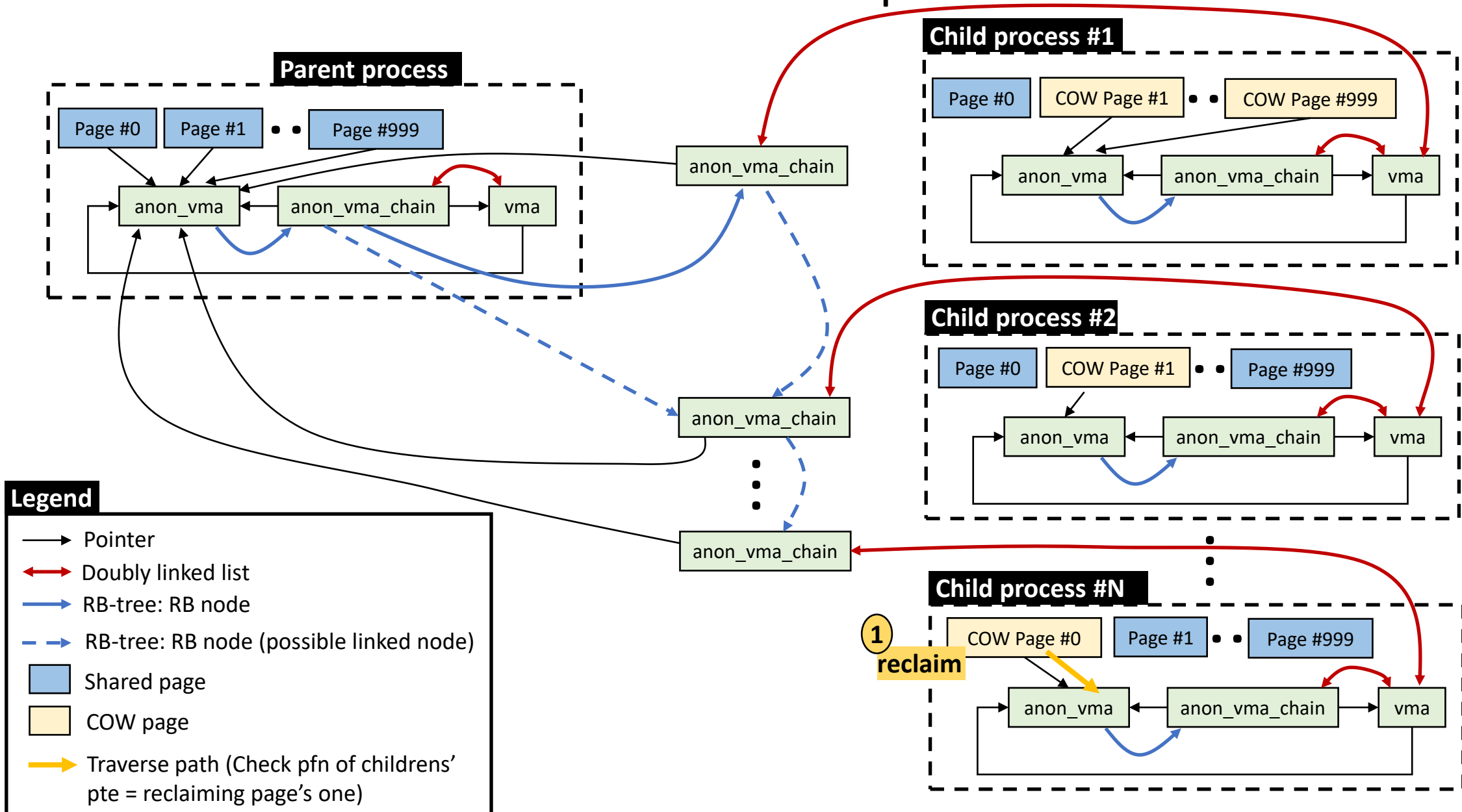
2.6.34 or later kernel – parent/child processes interconnection



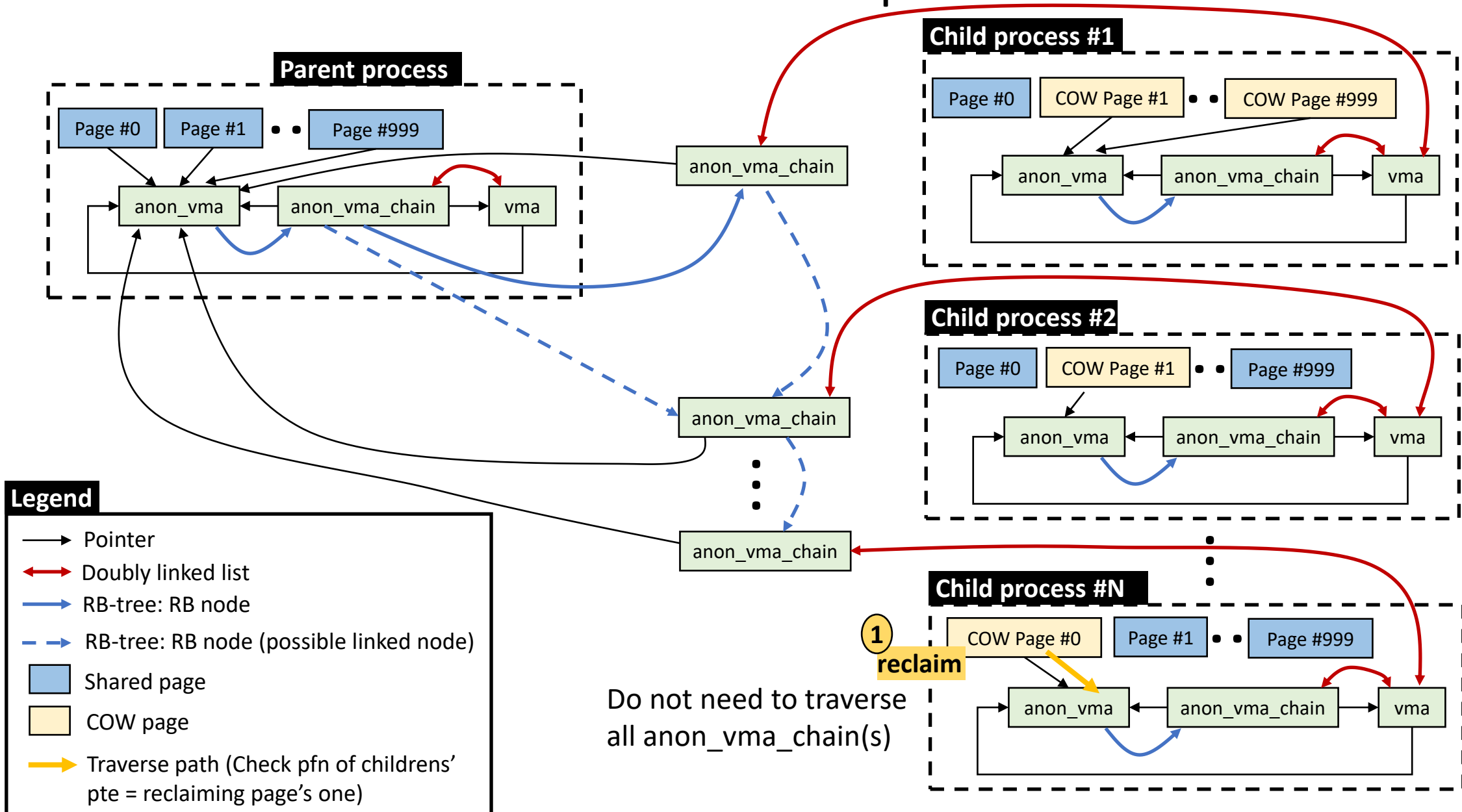
2.6.34 or later kernel: example 1



2.6.34 or later kernel: example 2



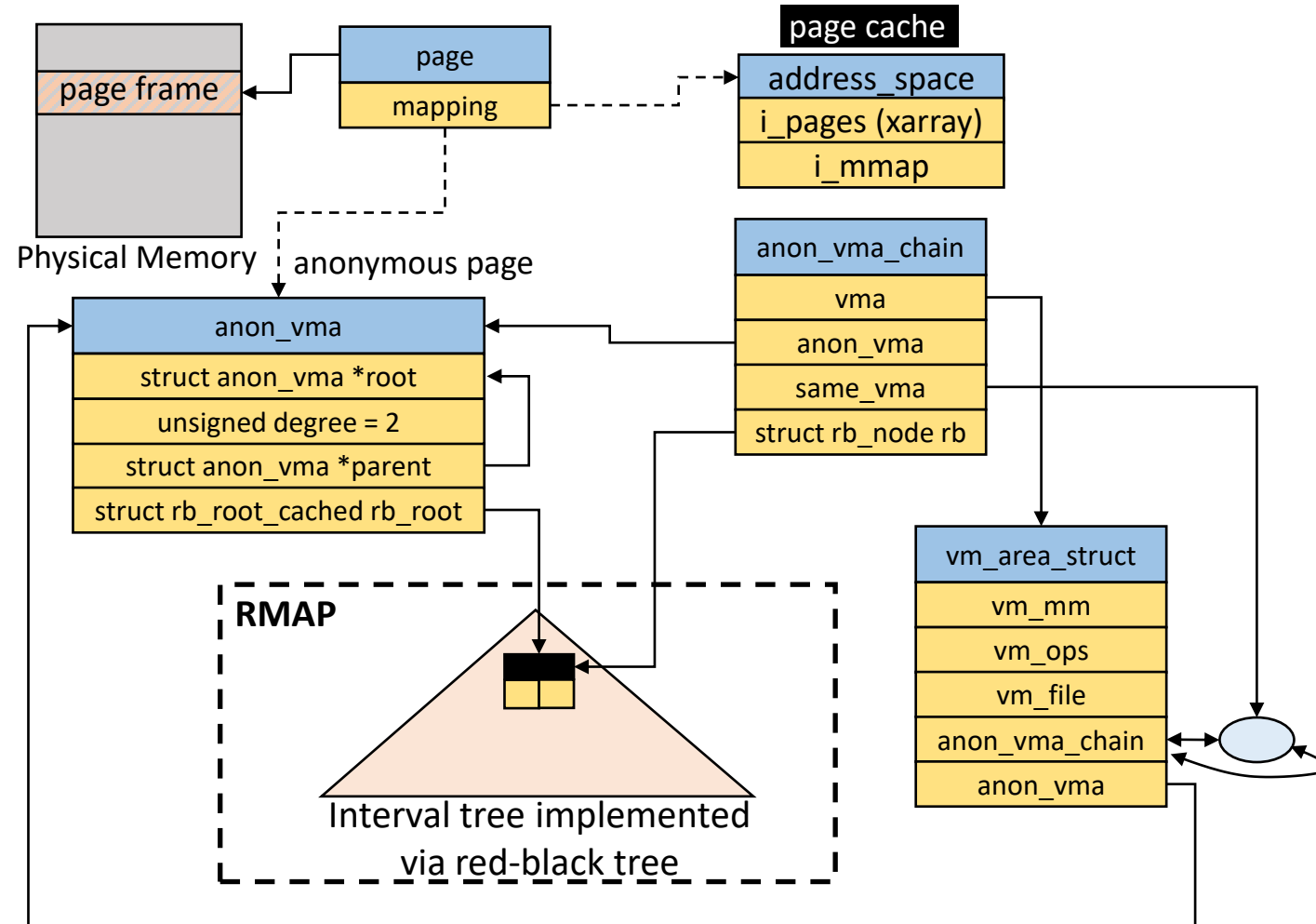
2.6.34 or later kernel: example 2



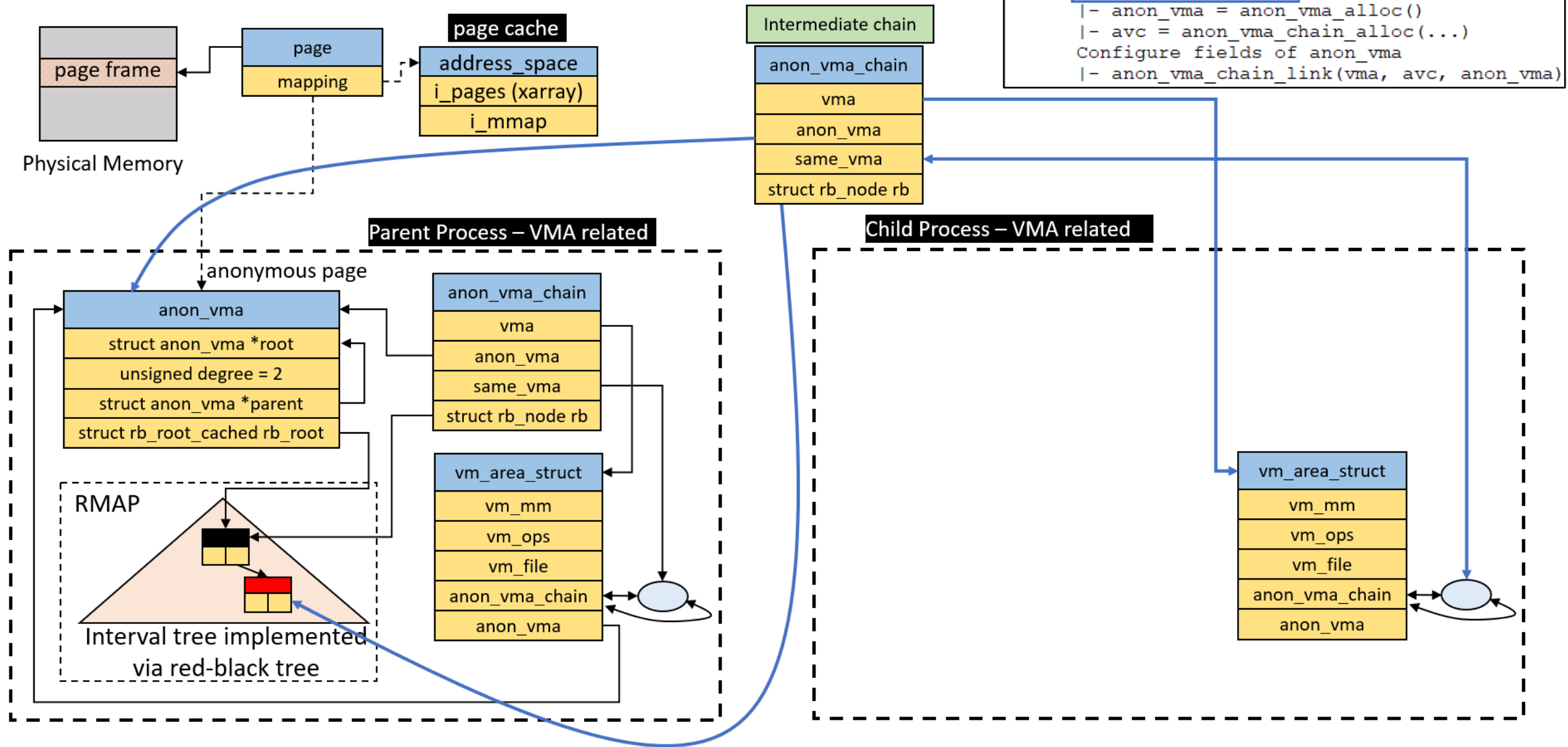
Implementation Detail

1. How/when to construct/link anon_vma, anon_vma_chain and vm_area_struct
 - A. Let's start from fork()
2. COW - Detail

anon_vma, anon_vma_chain & vm_area_struct - Detail



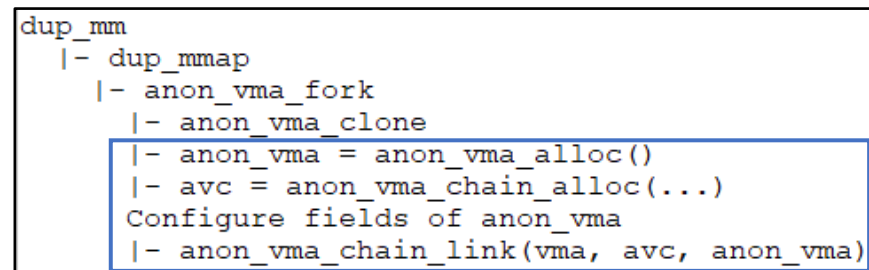
fork() – anon_vma_clone()



fork() – anon_vma_clone()

```
(gdb) bt
#0  anon_vma_clone (dst=dst@entry=0xffff88810493ec38,
    src=src@entry=0xffff88810493e8a0)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/mm/rmap.c:302
#1  0xffffffff810a112d in anon_vma_fork (vma=vma@entry=0xffff88810493ec38,
    pvma=pvma@entry=0xffff88810493e8a0)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/mm/rmap.c:345
#2  0xffffffff8102c630 in dup_mmap (mm=mm@entry=0xffff8881000e4700,
    oldmm=oldmm@entry=0xffff8881000e4380)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/kernel/fork.c:55
0
#3  0xffffffff8102c7b2 in dup_mm (oldmm=0xffff8881000e4380,
    tsk=0xffff888104969200)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/kernel/fork.c:13
60
#4  0xffffffff8102d9c2 in copy_mm (tsk=0xffff888104969200,
    clone_flags=18874368)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/arch/x86/include
    /asm/current.h:15
#5  copy_process (pid=pid@entry=0x0, trace=trace@entry=0, node=node@entry=-1,
    args=args@entry=0xffffc9000008feb8)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/kernel/fork.c:20
97
#6  0xffffffff8102df68 in kernel_clone (args=args@entry=0xffffc9000008feb8)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/kernel/fork.c:24
62
#7  0xffffffff8102e220 in __do_sys_clone (clone_flags=<optimized out>,
    newsp=<optimized out>, parent_tidptr=<optimized out>,
    child_tidptr=<optimized out>, tls=<optimized out>)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/kernel/fork.c:25
79
#8  0xffffffff8102e400 in __se_sys_clone (tls=<optimized out>,
    child_tidptr=<optimized out>, parent_tidptr=<optimized out>,
    newsp=<optimized out>, clone_flags=<optimized out>)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/kernel/fork.c:25
63
#9  __x64_sys_clone (regs=<optimized out>)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/kernel/fork.c:25
63
#10  0xffffffff81220b42 in do_syscall_64 (nr=<optimized out>,
    regs=0xffffc9000008ff58)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/arch/x86/entry/c
    ommon.c:46
#11  0xffffffff81400079 in entry_SYSCALL_64 ()
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/arch/x86/entry/e
    ntry_64.S:120
#12  0x0000000000000000 in ?? ()
```

```
dup_mm
|- dup_mmap
  |- anon_vma_fork
    |- anon_vma_clone
      |- anon_vma = anon_vma_alloc()
      |- avc = anon_vma_chain_alloc(...)
      Configure fields of anon_vma
      |- anon_vma_chain_link(vma, avc, anon_vma)
```



fork() – anon_vma_fork()

(gdb) bt

```
#0  anon_vma_fork (vma=vma@entry=0xffff88810493ec38,
    pvma=pvma@entry=0xffff88810493e8a0)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/mm/rmap.c:377
#1  0xffffffff8102c630 in dup_mmmap (mm=mm@entry=0xffff8881000e4700,
    oldmm=oldmm@entry=0xffff8881000e4380)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/kernel/fork.c:55
0
#2  0xffffffff8102c7b2 in dup_mm (oldmm=0xffff8881000e4380,
    tsk=0xffff888104969200)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/kernel/fork.c:13
60
#3  0xffffffff8102d9c2 in copy_mm (tsk=0xffff888104969200,
    clone_flags=18874368)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/arch/x86/include
/asm/current.h:15
#4  copy_process (pid=pid@entry=0x0, trace=trace@entry=0, node=node@entry=-1,
    args=args@entry=0xffffc9000008feb8)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/kernel/fork.c:20
97
#5  0xffffffff8102df68 in kernel_clone (args=args@entry=0xffffc9000008feb8)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/kernel/fork.c:24
62
#6  0xffffffff8102e220 in __do_sys_clone (clone_flags=<optimized out>,
    newsp=<optimized out>, parent_tidptr=<optimized out>,
    child_tidptr=<optimized out>, tls=<optimized out>)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/kernel/fork.c:25
79
#7  0xffffffff8102e400 in __se_sys_clone (tls=<optimized out>,
    child_tidptr=<optimized out>, parent_tidptr=<optimized out>,
    newsp=<optimized out>, clone_flags=<optimized out>)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/kernel/fork.c:25
63
#8  __x64_sys_clone (regs=<optimized out>)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/kernel/fork.c:25
63
#9  0xffffffff81220b42 in do_syscall_64 (nr=<optimized out>,
    regs=0xffffc9000008ff58)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/arch/x86/entry/c
ommon.c:46
#10 0xffffffff81400079 in entry_SYSCALL_64 ()
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/arch/x86/entry/e
ntry_64.S:120
#11 0x0000000000000000 in ?? ()
```

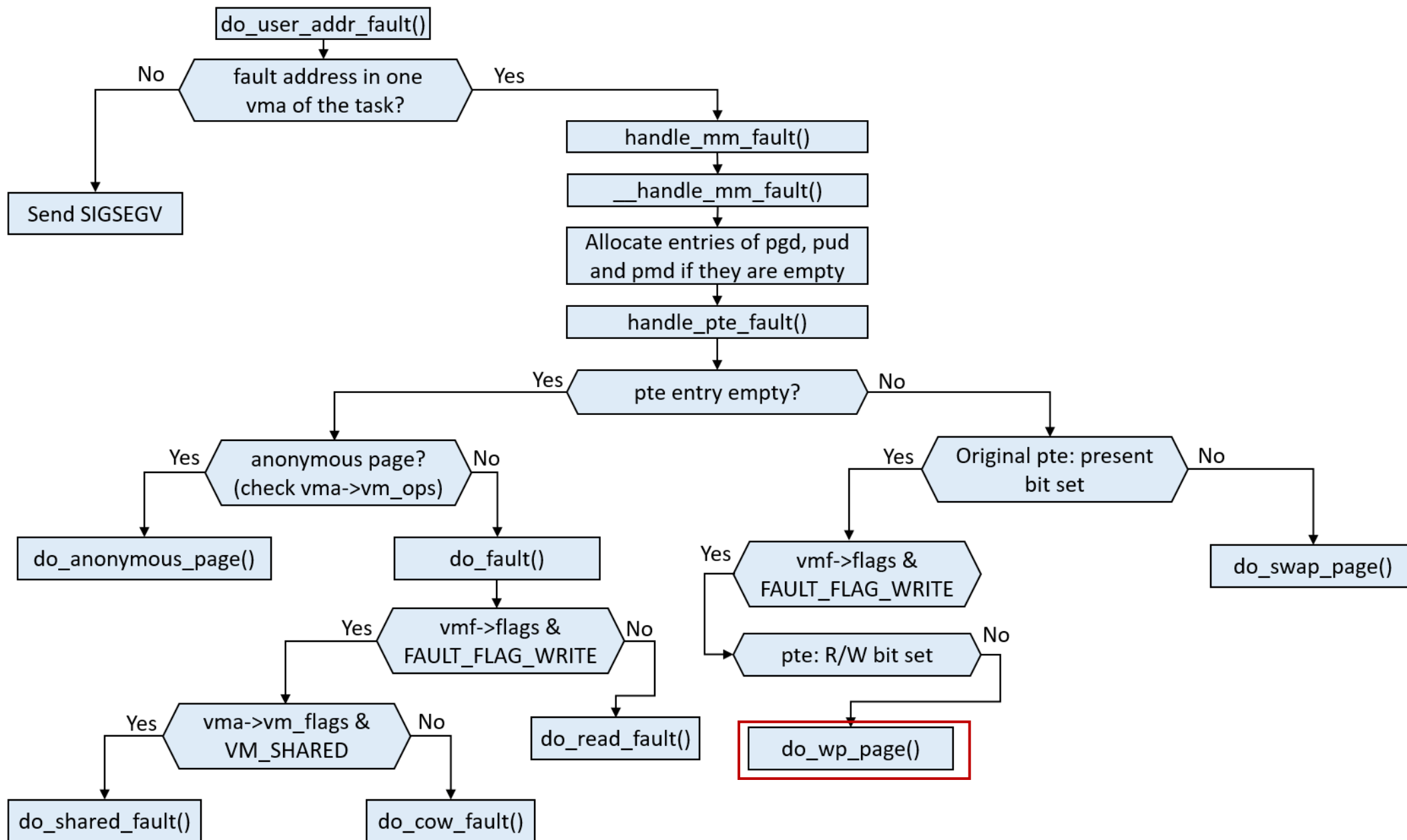
```
int anon_vma_fork(struct vm_area_struct *vma, struct vm_area_struct *pvma)
{
+-- 44 lines: struct anon_vma_chain *avc;-----
    vma->anon_vma = anon_vma;
    anon_vma_lock_write(anon_vma);
    anon_vma_chain_link(vma, avc, anon_vma);
    anon_vma->parent->degree++;
    anon_vma_unlock_write(anon_vma);

    return 0;
+-- 6 lines: out_error_free_anon_vma:-----
}
```

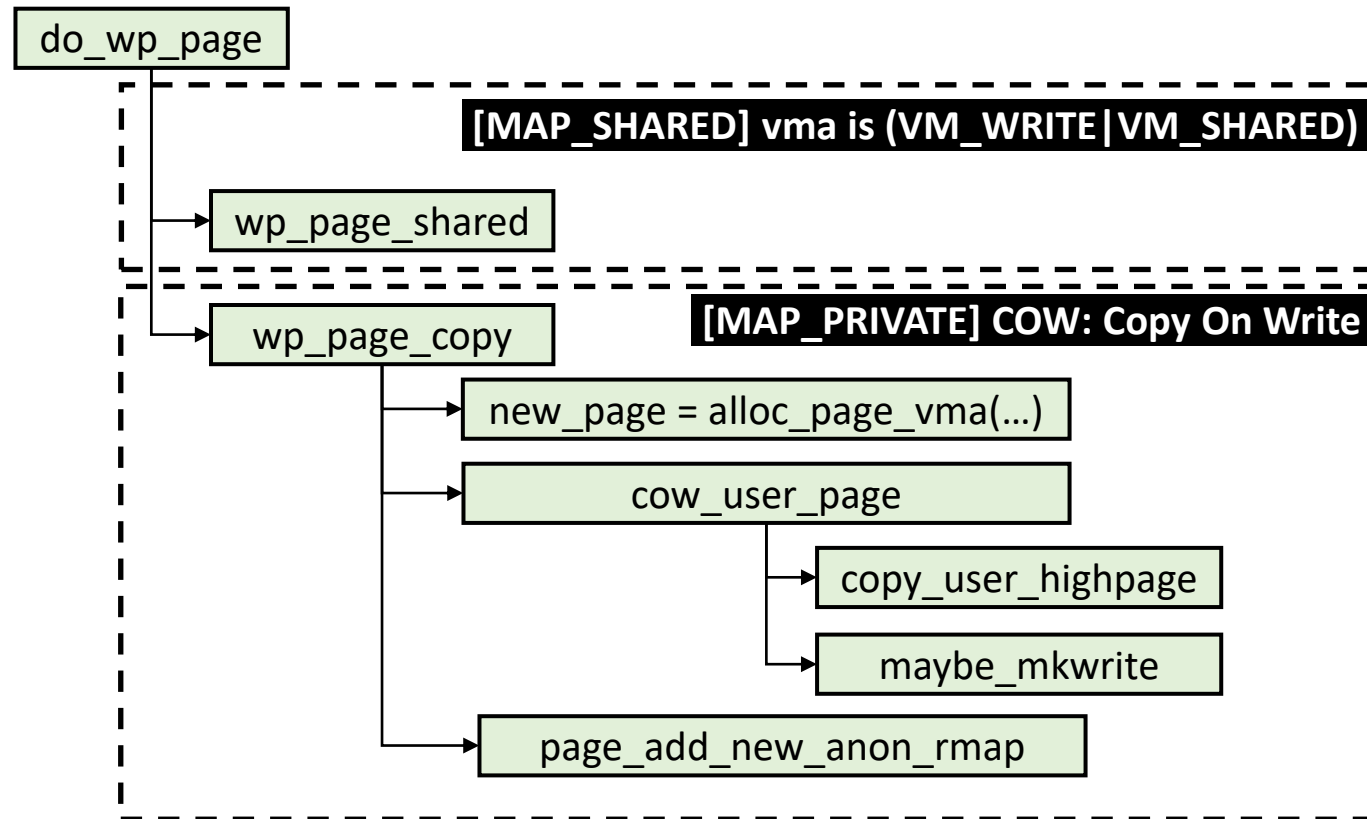
mm/rmap.c

377,1-8

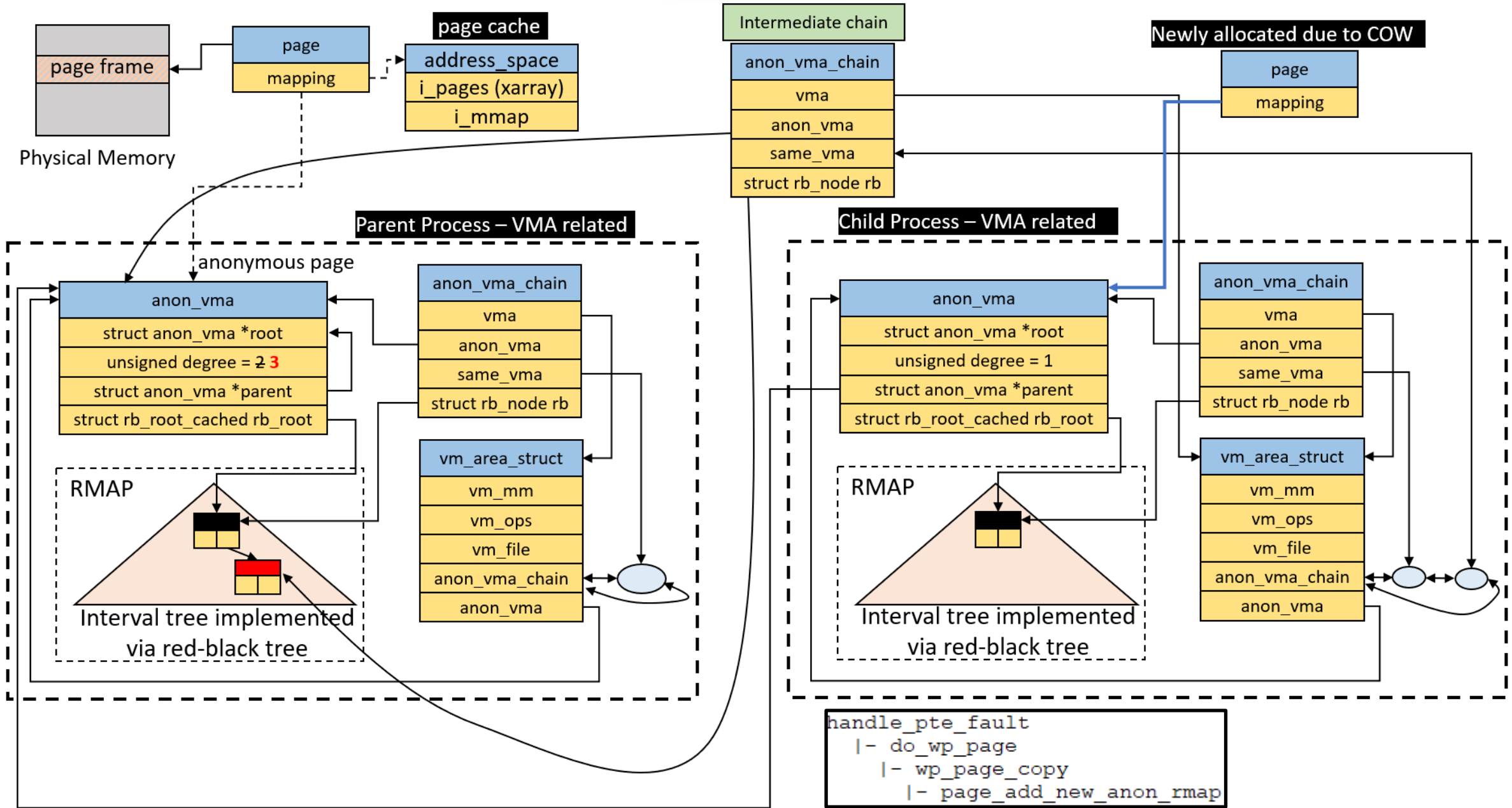
fork(): COW $\Rightarrow$ write fault (write-protected fault)



fork(): COW → write fault (write-protected fault)



fork(): COW - page_add_new_anon_rmap()



Child Process: COW

```
(gdb) bt
#0  wp_page_copy (vmf=vmf@entry=0xffffc90000087e38)
   at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/mm/memory.c:2830
#1  0xffffffff810930b1 in do_wp_page (vmf=vmf@entry=0xffffc90000087e38)
   at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/mm/memory.c:3147
#2  0xffffffff81094783 in handle_pte_fault (vmf=0xffffc90000087e38)
   at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/mm/memory.c:4405
#3  __handle_mm_fault (flags=597, address=140737354108928, vma=<optimized out>)
   at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/mm/memory.c:4522
#4  handle_mm_fault (vma=<optimized out>,
   address=address@entry=140737354108928, flags=flags@entry=597,
   regs=regs@entry=0xffffc90000087f58)
   at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/mm/memory.c:4620
#5  0xffffffff81025107 in do_user_addr_fault (
   regs=regs@entry=0xffffc90000087f58, hw_error_code=hw_error_code@entry=7,
   address=address@entry=140737354108928)
   at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/arch/x86/mm/fault.c:1393
#6  0xffffffff8122232b in handle_page_fault (address=140737354108928,
   error_code=7, regs=0xffffc90000087f58)
   at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/arch/x86/mm/fault.c:1450
#7  exc_page_fault (regs=0xffffc90000087f58, error_code=7)
   at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/arch/x86/mm/fault.c:1506
#8  0xffffffff81400a6b in asm_exc_page_fault ()
   at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-5.11/arch/x86/include
```

```
/*
 * Page fault error code bits:
 *
 * bit 0 == 0: no page found      1: protection fault
 * bit 1 == 0: read access        1: write access
 * bit 2 == 0: kernel-mode access 1: user-mode access
 * bit 3 == 0: use of reserved bit detected
 * bit 4 == 0: fault was an instruction fetch
 * bit 5 == 0: protection keys block access
 * bit 15 == 0: SGX MMU page-fault
 */
enum x86_pf_error_code {
    X86_PF_PROT      = 1 << 0,
    X86_PF_WRITE     = 1 << 1,
    X86_PF_USER      = 1 << 2,
    X86_PF_RSVD      = 1 << 3,
    X86_PF_INSTR     = 1 << 4,
    X86_PF_PK        = 1 << 5,
    X86_PF_SGX       = 1 << 15,
};
```

```
(gdb) p /x *vmf
$138 = {
  vma = 0xffff88810493e678,
  flags = 0x255,
  gfp_mask = 0xcc0,
  pgoff = 0x7ffff7ff9,
  address = 0x7ffff7ff9000,
  pmd = 0xffff888104946df8,
  pud = 0xffff888104949ff8,
  orig_pte = {
    pte = 0x8000000240317045
  },
  cow_page = 0x0,
  page = 0xffffea000900c5c0,
  pte = 0xffff888104943fc8,
  ptl = 0xffff8881000e4764,
  prealloc_pte = 0x0
}
```

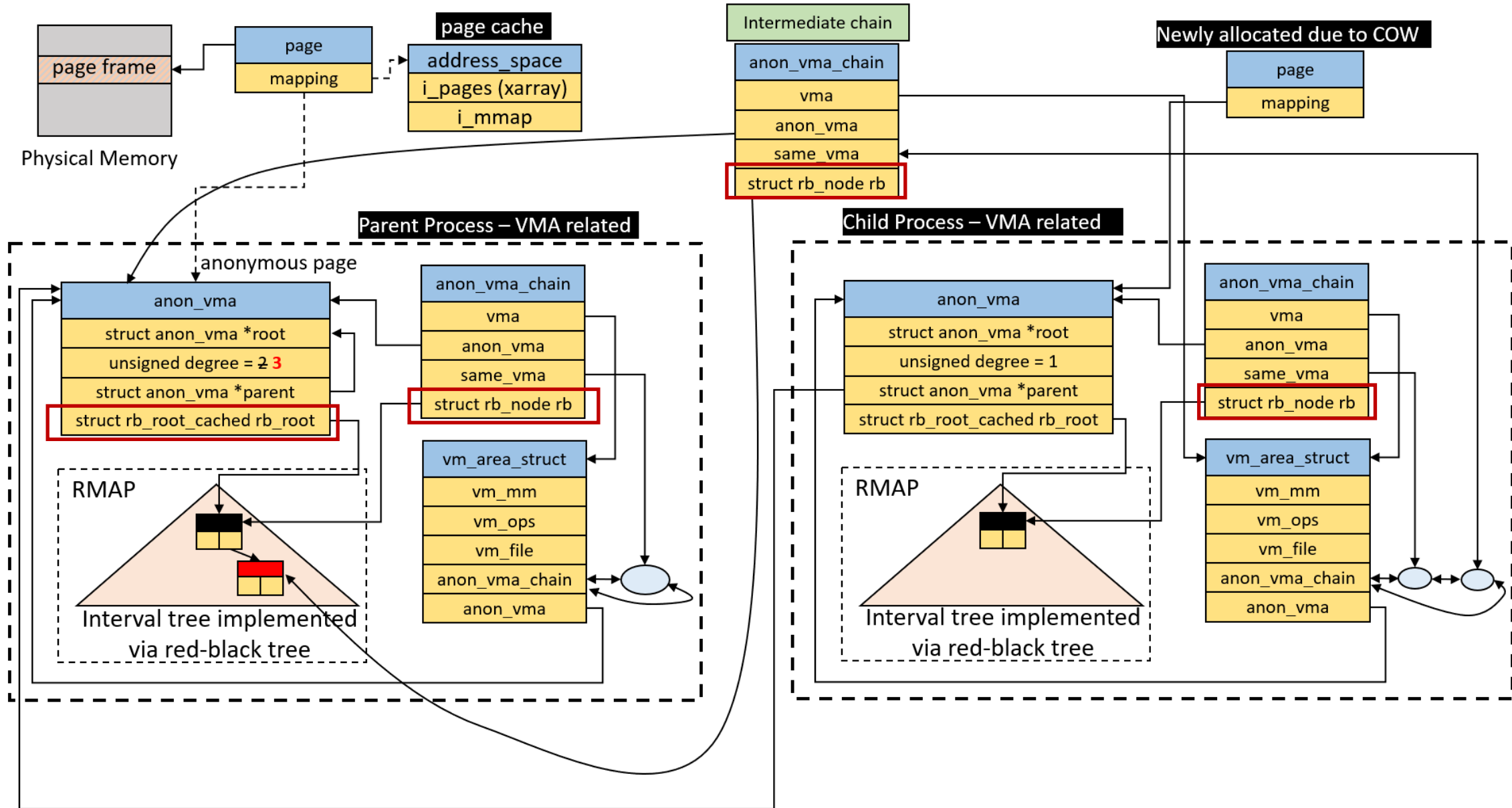
```
(gdb) p /x *vmf->pte
$141 = {
  pte = 0x8000000240317045
}
```

Write fault

Table 4-20. Format of a Page-Table Entry that Maps a 4-KByte Page

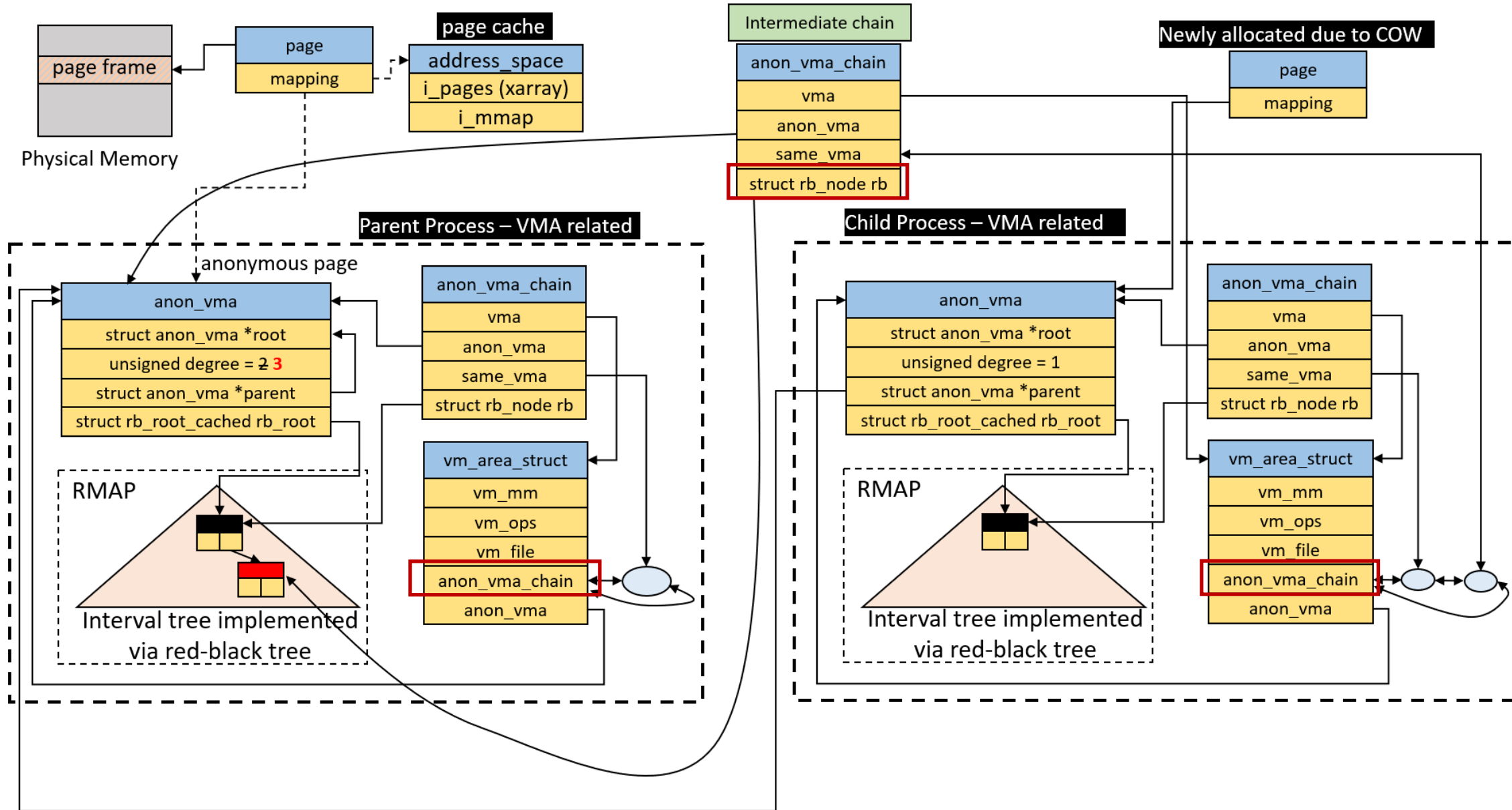
Bit Position(s)	Contents
0 (P)	Present; must be 1 to map a 4-KByte page
1 (R/W)	Read/write; if 0, writes may not be allowed to the 4-KByte page referenced by this entry (see Section 4.6)
2 (U/S)	User/supervisor; if 0, user-mode accesses are not allowed to the 4-KByte page referenced by this entry (see Section 4.6)
3 (PWT)	Page-level write-through; indirectly determines the memory type used to access the 4-KByte page referenced by this entry (see Section 4.9.2)

rb_root & rb – When/who to use?



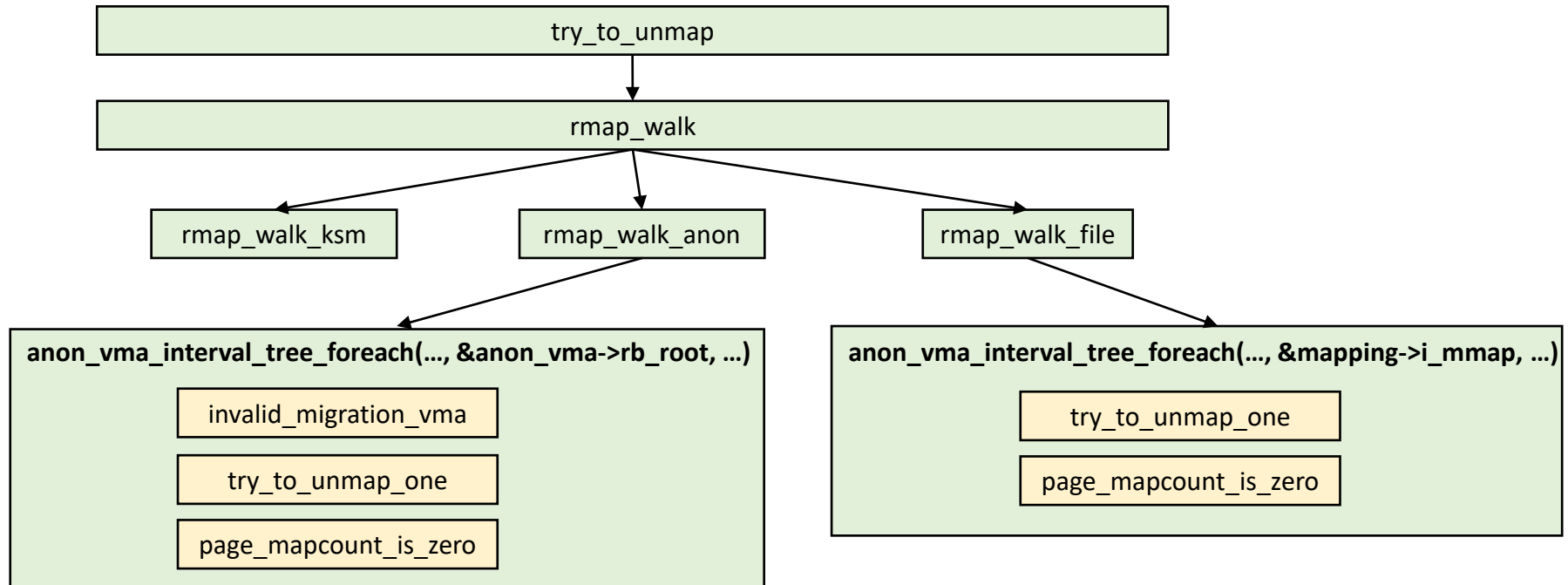
Interval tree traversal (implemented via red-black tree) for reverse mapping

struct list_head anon_vma_chain – When/who to use?



Remove VMA: check unlink_anon_vmas()

rmap: page reclaiming – try_to_unmap()



Reference

- Understanding the Linux Kernel, 3rd Edition
- [【原创】（十五）Linux内存管理之RMAP](#)
- 奔跑吧 Linux 内核